Incident Management

one commander, mitigate first, learn after

Watch the 4-min video

In one line: classify by severity, put one incident commander in charge with a comms lead and responders, stop the bleeding before finding the root cause, update on a fixed rhythm, then run a blameless postmortem whose action items have owners and due dates.

1 Ten engineers, nobody in charge

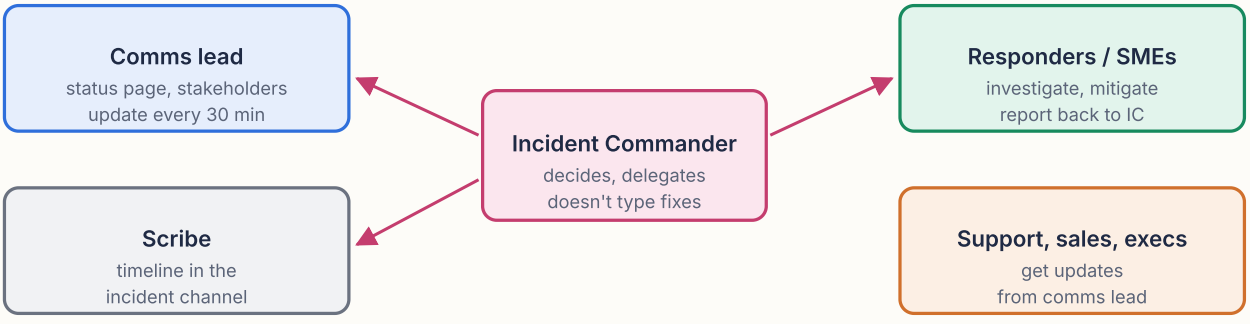
A major outage starts. Ten engineers jump into the same chat. Three restart different services at once. Nobody tells customers anything, and support is flooded. Everyone is busy, but nobody is in charge. An incident process exists to replace that chaos with roles, priorities and a rhythm. Many teams base theirs on the Incident Command System adapted by Google SRE and PagerDuty.

2 Severity levels

Level	Meaning	Example	Response
SEV1	critical user-facing outage	checkout down for everyone	page now, IC assigned, updates every 30 min, status page
SEV2	major feature degraded or some users affected	search slow in one region	page on-call, regular updates
SEV3	minor impact, workaround exists	CSV export fails, retry works	ticket, business hours

Severity decides who is paged, how fast, and how often you communicate. Exact definitions vary by company; what matters is that they are written down and anyone can declare one. When unsure, declare higher and downgrade later: it's cheap to stand down.

3 Roles



one person in charge → no conflicting actions; roles hand over in shifts

Figure 1. The incident commander coordinates; others act and communicate. In a small team one person may hold two roles, but never IC and hands-on fixer in a SEV1.

The **incident commander** (IC) makes decisions, delegates tasks and keeps everyone focused; they don't debug. The **communications lead** posts updates to stakeholders and the status page. **Subject-matter**

experts investigate and act, reporting back to the IC. A **scribe** keeps the timeline. For long incidents, roles hand over in shifts with an explicit handover, so nobody decides things after 18 hours awake.

4 Mitigation before resolution

The first goal is to **stop the bleeding**, not to understand everything. Roll back the deploy (lesson 72), turn off the feature flag (lesson 76), shift traffic to another region (lesson 15), or scale up. Root cause can wait until users stop hurting (lesson 69). Practise rollbacks in calm times so they're fast and boring when it matters.



Figure 2. The incident lifecycle. Mitigation ends the user pain; the postmortem and tracked actions make the next incident less likely.

5 Communication rhythm

Every **30 minutes** for a SEV1 (or as your policy says) the comms lead posts: what is known, what is being tried, and when the next update comes, *even if nothing changed*. One incident channel is the record. Silence makes customers assume the worst; internal teams (support, sales) need updates too, so they know what to tell people.

6 Blameless postmortems

A postmortem asks **how the system allowed the failure**, not who made the mistake. People who fear blame hide information, and the same failure returns. It records the timeline, impact (users, duration, error budget spent), contributing factors and **action items, each with an owner and a due date**. Track them to done, or the postmortem was just paperwork.

WORKED EXAMPLE: A BAD RELEASE, ROLLED BACK

v2.4 goes out and checkout errors spike. Someone declares **SEV1**; an IC takes charge and a comms lead updates the status page. Instead of debugging the new code live, the IC orders a **rollback to v2.3**. Errors recover within **5 minutes**, and the status page says so. Next day, the blameless postmortem finds a missing integration test for the payment path. Action items: add the test (owner: payments team, due Friday) and **automatic rollback on error spikes** in the deploy pipeline (owner: platform team).

COMMON MISTAKES

- The most senior engineer acting as IC *and* debugging, so nobody coordinates.
- Hunting the root cause while users are still failing, when a rollback was available.
- No updates for an hour because "nothing has changed".
- Postmortems that name a person, or action items with no owner and no date.

7 Key terms

SEV level: severity class deciding response speed and scope

Comms lead: owns stakeholder and status-page updates

Blameless postmortem: review of how the system allowed the failure

Incident commander: the one coordinator and decision maker

Mitigation: action that stops user impact without a full fix

Action item: owned, dated fix from a postmortem

8 Interview questions

Q1. Describe an incident process. BASIC

Classify by severity, assign an incident commander, a comms lead and responders, mitigate first (rollback, flag off, failover), communicate on a regular rhythm, then run a postmortem.

Q2. What is a blameless postmortem? BASIC

A review of how the system allowed the failure rather than who to blame, recording timeline, impact and contributing factors, ending in action items with owners and due dates.

Q3. Errors spiked right after a deploy. The author wants 10 minutes to find the bug. What does the IC do? SCENARIO

Roll back first if it's safe: it ends the impact within minutes. Debugging continues afterwards against the bad version in a non-production environment.

Q4. You're unsure if it's a SEV1 or SEV2. Which do you declare? SCENARIO

The higher one: over-declaring costs a little attention; under-declaring delays help and communication. Downgrade when impact is clear.

Q5. Why shouldn't the IC fix things themselves? TRICKY

Hands-on work narrows attention; nobody then tracks parallel efforts, prevents conflicting actions or decides when to escalate and communicate.

Q6. Why can a blameless culture improve reliability? TRICKY

People report mistakes and near-misses honestly, so the real contributing factors are found and fixed instead of hidden.

Next: SD-N-013 · Capacity Planning backendroadmap.com